CENTRO UNIVERSITÁRIO SANTO AGOSTINHO- UNIFSA

PRÓ-REITORIA DE ENSINO

NÚCLEO DE APOIO PEDAGÓGICO-NUAPE

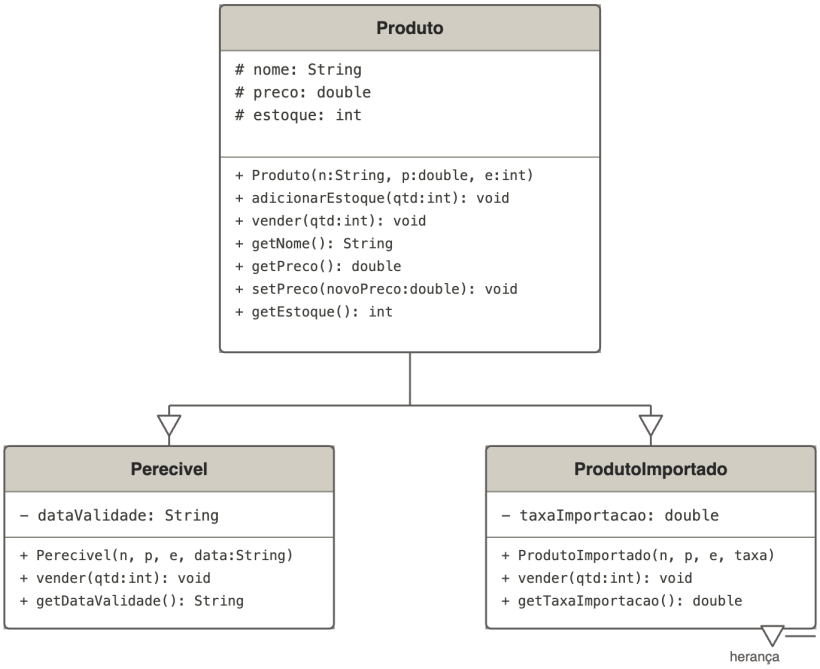
COORDENAÇÃO DO CURSO DE ENGENHARIA DE SOFTWARE

DISCIPLINA: PROGRAMAÇÃO ORIENTADA À OBJETOS

PROFA: MA. HELOISA GUIMARÃES

Desafio de Auditoria: Sistema de Gestão Supermercado

Sua equipe foi contratada para dar manutenção em um sistema de gestão de uma grande rede de supermercados. A equipe de planejamento inicialmente estruturou o módulo do projeto orientado à objetos que cuida dos produtos da seguinte forma:



O estagiário responsável pela implementação "esqueceu" as regras básicas de Programação Orientada a Objetos e colocou vários bugs no sistema. A aplicação compila com avisos, mas está cheio de falhas conceituais que podem causar prejuízos imensos ao mercado. A tarefa da sua equipe é identificar, documentar e corrigir os erros de POO antes da implementação oficial.

PARTE 1: Auditoria "Analógica" (Teste de mesa)

Analise o código abaixo e estruture uma **Folha de Ocorrências**:

1. **Mapeamento de Bugs:** Localize ao menos 8 erros relacionados a:
    - **Encapsulamento:** Atributos que deveriam estar protegidos, mas estão expostos.
    - **Herança:** Problemas na comunicação entre Classe Pai e Classe Filha.
    - **Construtores:** Objetos sendo criados de forma incompleta ou errada.
2. **Rastreio de Memória:** Desenhe como ficaria o objeto `leite` na memória. Quais atributos ele possui? Quais ele herdou?
3. **Justificativa Técnica:** Para cada erro, escreva qual conceito de POO foi violado.

PARTE 2: Implementação e Expansão (Netbeans)

Após a correção coletiva dos erros encontrados, sua equipe deve realizar as seguintes missões no sistema:

Missão 1: Blindagem de Dados

Implemente métodos `getters` e `setters` para todos os atributos. Observem que métodos como o `setPreco` e o `setEstoque` **não podem** aceitar valores negativos. Se o usuário tentar inserir um valor inválido, o sistema deve exibir uma mensagem de erro e manter o valor original.

Missão 2: Especialização

Crie uma nova classe chamada `ProdutoImportado`.

- Ela deve herdar de `Produto`.
- Deve possuir um atributo adicional: `taxaImportacao` (float).
- O método `vender` deve ser sobrescrito para que, ao ser chamado, ele exiba o preço final já com a taxa aplicada.

Missão 3: O Gestor de Vendas

Crie uma classe `Venda`. Ela deve ter um método chamado `processarItem(Produto p, int quantidade)`.

- Este método deve receber qualquer tipo de produto (Normal, Perecível ou Importado).
- Ele deve calcular o valor total (quantidade * preço) e exibir o recibo no console.
- **Desafio:** O método deve conseguir chamar o `vender()` correto de cada objeto.

Obs: Diferenciem **Sobrecarga** (mesmo nome, parâmetros diferentes) de **Sobrescrita** (mesmo nome, comportamento diferente na herança).

Missão 4: Projeto de Software

- Atualize o diagrama de classes do projeto com base nas modificações.

```java
// ================== CLASSE BASE ==================
class Produto {
    private String nome;
    private float preco;
    private int estoque;

    public Produto(String n, float p, int e) {

    }

    public void adicionarEstoque(int qtd) {
        estoque += qtd;
    }

    public void vender(int qtd) {
        estoque -= qtd;
        System.out.println("Vendido: " + qtd + " unidades.");
    }
}
// ================== SUBCLASSE (HERANÇA) ==================
class Perecivel extends Produto {
    private String dataValidade;

    public Perecivel(String n, float p, int e, String data) {
        this.nome = n;
        this.preco = p;
        this.dataValidade = data;
    }

    public void vender() {
        System.out.println("Verificando validade...");
        if (estoque > 0) {
            System.out.println("Item perecível vendido.");
        }
    }
}
// ================== EXECUÇÃO ==================
public class Supermercado {
    public static void main(String[] args) {
        Produto p1 = new Produto("Arroz", 25.50);
        p1.preco = -10.0;
        Perecivel leite = new Perecivel("Leite", 5.0, 10, "10/05/2026");
        leite.vender(2);
    }
}
```

Folha de Ocorrências

ID	Localização (Classe/Linha)	Descrição do Erro	Conceito de POO Violado
01			
02			
03			
04			
05			
06			
07			
08			